Algorithms and Development of Programmes for Biological Chemistry

Agata M. Kilar

Research Group Bioinformatics and Computational Biology
Department of Theoretical Chemistry
Währingerstrasse 17, 1090 Vienna, Austria

Summer 2026

Markov's motivation

What is the probability of a letter being a vowel or a consonant, given the previous letter?

A Pushkin fragment

"He was too young to have
been blighted
by the cold world's corrupt
finesse."

**from Eugene Onegin*

Markov's setup

w a s t o o y o u n g t o h a v e b e e n b l i g h t e d

Markov took about **10,000 characters** and treated them simply as a string of letters, ignoring the meaning.

w a s t o o y o u n g t o h a v e b e e n b l i g h t e d

Markov counted how often he saw a **vowel** and how often he saw a **consonant**.

Observed adjacent pairs

w a s t o o y o u n g t o h a v e b e e n b l i g h t e d

Then he broke the string into **overlapping pairs**, which gives four possible combinations:

Ⓥ Ⓥ *VV*: observed about **6**%

Ⓒ Ⓒ *CC*: observed about **19**%

Ⓥ Ⓒ *VC*: observed about **37.5**%

Ⓒ Ⓥ *CV*: observed about **37.5**%

What independence would predict

$VV \approx 6\%$	$VV = 0.43 \times 0.43 \approx 18\%$
$CC \approx 19\%$	$CC = 0.57 \times 0.57 \approx 32\%$
$VC \approx 37.5\%$	$VC = 0.43 \times 0.57 \approx 25\%$
$CV \approx 37.5\%$	$CV = 0.57 \times 0.43 \approx 25\%$
Observed in the phrase	**If letters were independent**

The mismatch suggests that consecutive letters are **not independent**.

A simple two-state Markov model

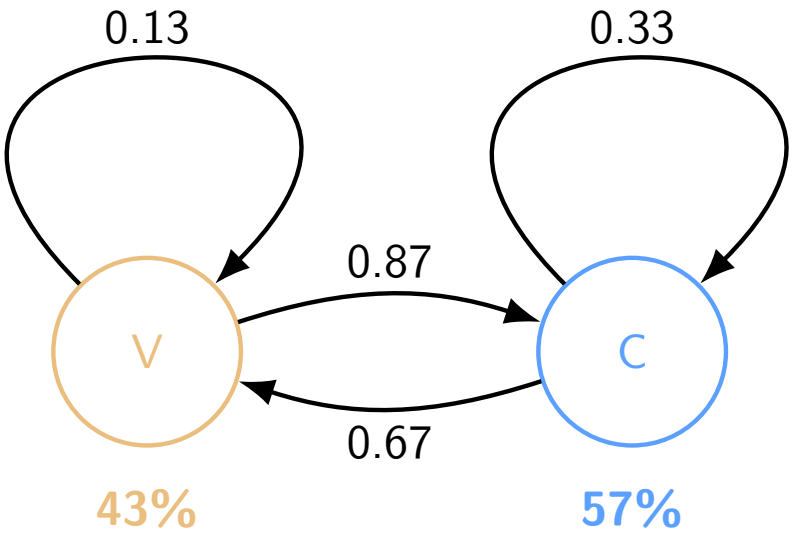
From the observed pair frequencies:

$$P(V \to V) \approx \frac{0.06}{0.43} = 0.13$$

$$P(V \to C) = 1 - 0.13 = 0.87$$

$$P(C \to V) \approx \frac{0.375}{0.57} \approx 0.67$$

$$P(C \to C) = 1 - 0.67 = 0.33$$

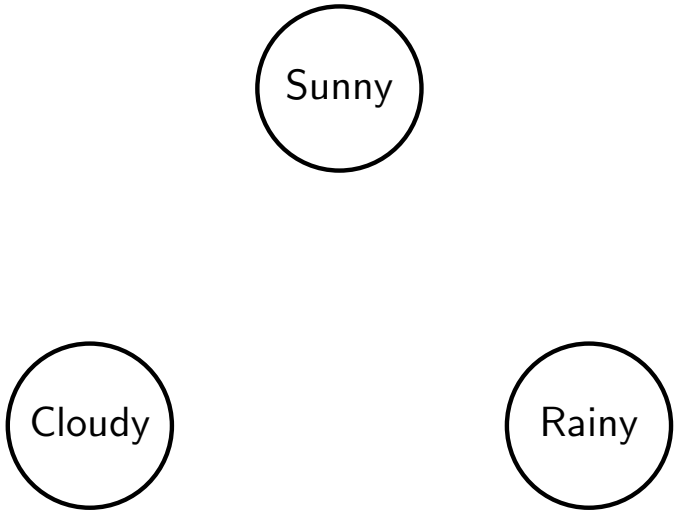


Three basic ingredients

1. **States** 2. **Transitions** 3. **Probabilities**

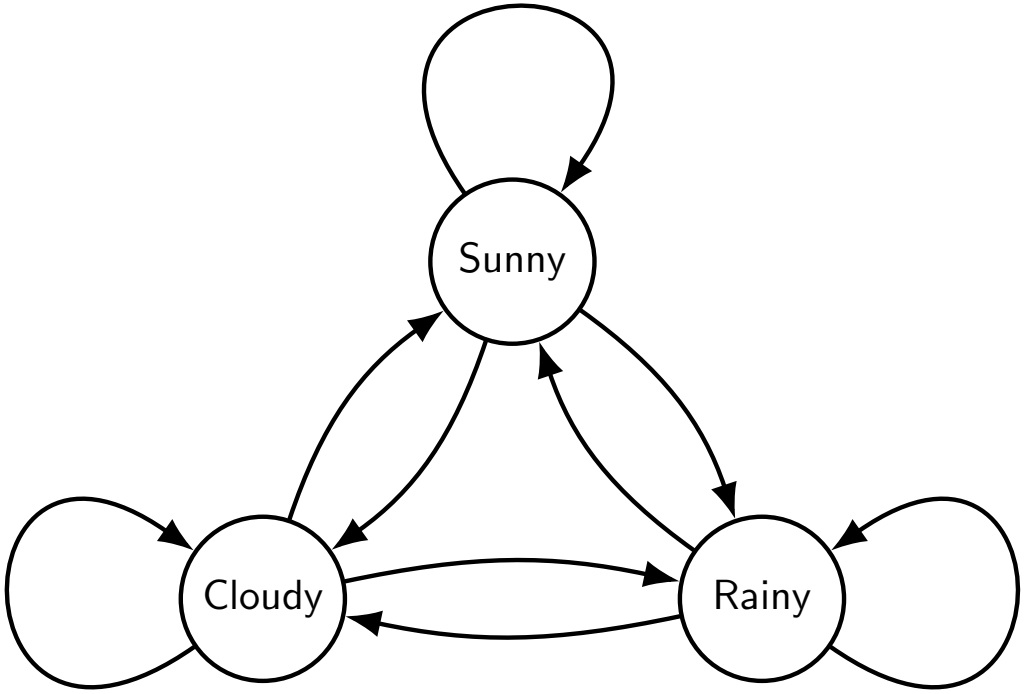
States

The possible configurations of the system:



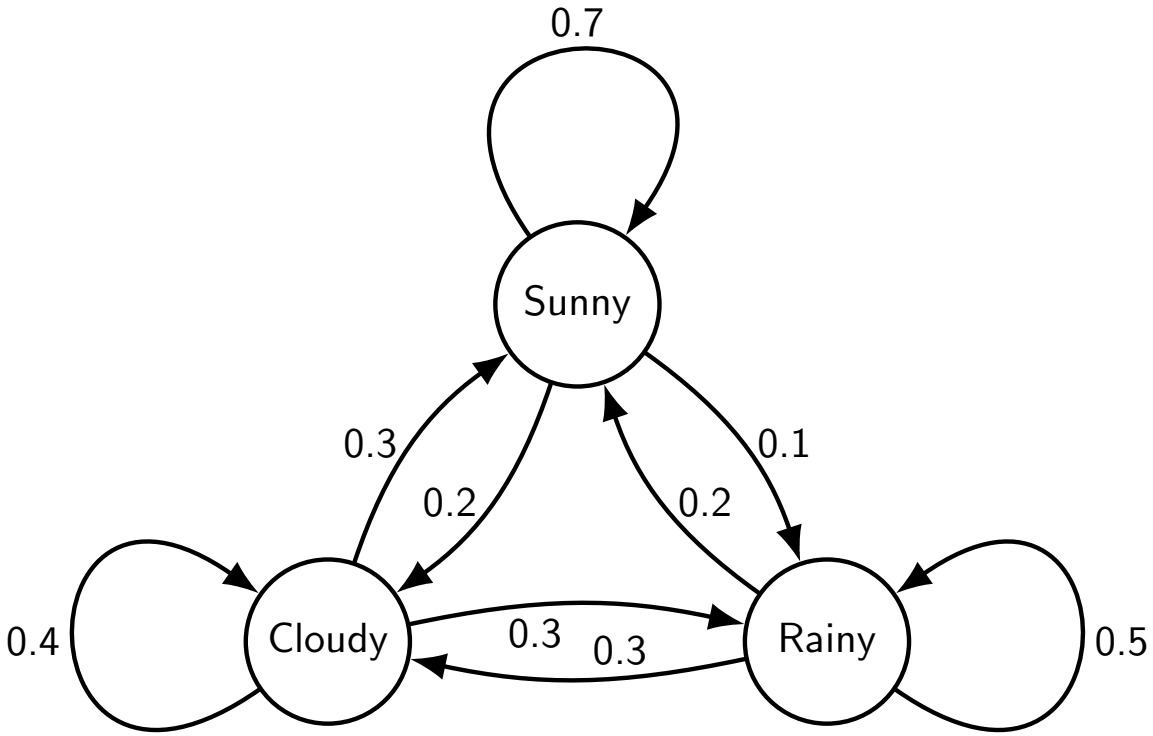
Transitions

How the system moves between states:



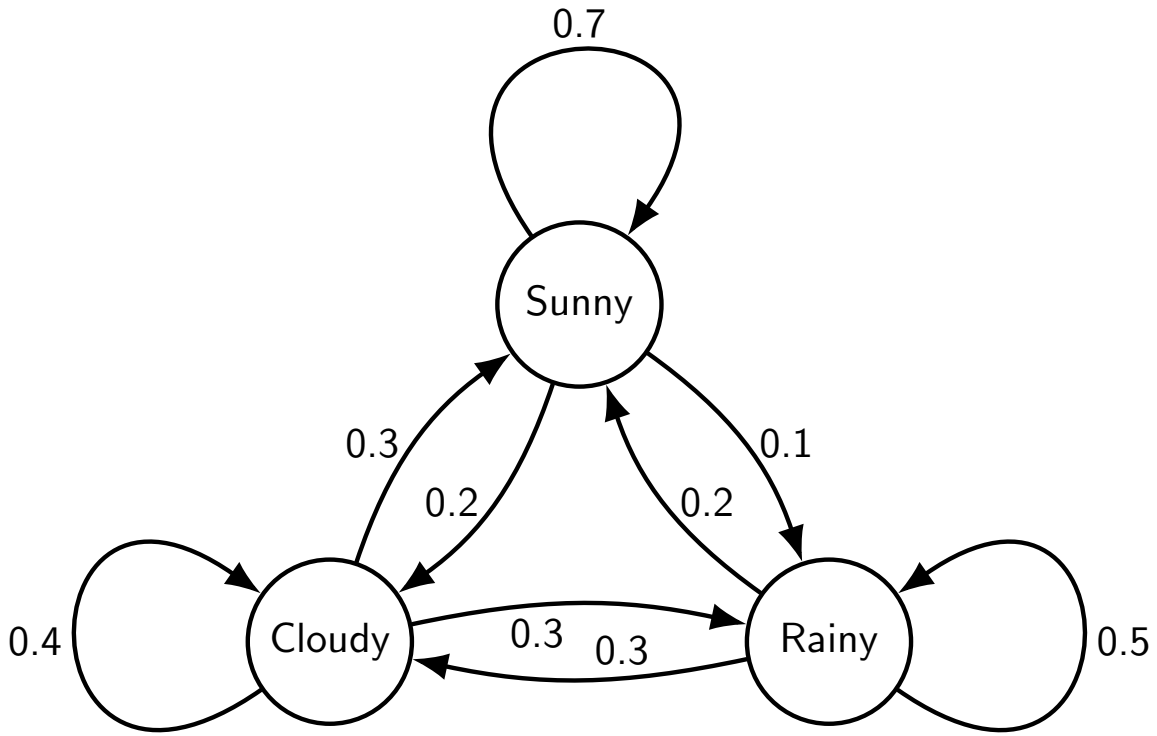
Probabilities

Probabilities tell us how likely each move is:



For each state, the probabilities of all outgoing transitions must sum to 1.

Transition matrix



	S	C	R
S	0.7	0.2	0.1
C	0.3	0.4	0.3
R	0.2	0.3	0.5

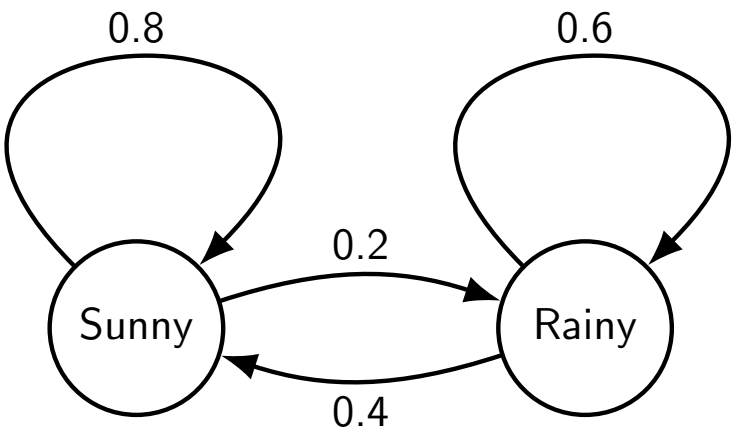
$$P_{ij} = P(X_{n+1} = j \mid X_n = i)$$

A practical example

Weather transitions over time

- We use only two states: **Sunny** and **Rainy**.
- The current weather determines the probabilities for tomorrow.
- Then we ask what happens after 2, 5, or many more days.

Weather model example



	S	R
S	0.8	0.2
R	0.4	0.6

If today is Sunny, tomorrow is Sunny with probability 0.8.

If today is Rainy, it stays Rainy with probability 0.6.

Two days later if we start in Sunny

Possible 2-step paths from **Sunny**:

$S \to S \to S$ $\quad 0.8 \times 0.8 = \mathbf{0.64}$

$S \to S \to R$ $\quad 0.8 \times 0.2 = 0.16$

$S \to R \to S$ $\quad 0.2 \times 0.4 = \mathbf{0.08}$

$S \to R \to R$ $\quad 0.2 \times 0.6 = 0.12$

Add the paths that end in the same state:

$$P(\text{Sunny after 2 days}) = 0.64 + 0.08 = \mathbf{0.72}$$

$$P(\text{Rainy after 2 days}) = 0.16 + 0.12 = \mathbf{0.28}$$

So the first row of P^2 is:

$$[0.72 \;\; 0.28]$$

Two days later if we start in Sunny

Possible 2-step paths from **Sunny**:

$S \to S \to S$ $\quad 0.8 \times 0.8 = \mathbf{0.64}$

$S \to S \to R$ $\quad 0.8 \times 0.2 = 0.16$

$S \to R \to S$ $\quad 0.2 \times 0.4 = \mathbf{0.08}$

$S \to R \to R$ $\quad 0.2 \times 0.6 = 0.12$

Add the paths that end in the same state:

$$P(\text{Sunny after 2 days}) = 0.64 + 0.08 = \mathbf{0.72}$$

$$P(\text{Rainy after 2 days}) = 0.16 + 0.12 = \mathbf{0.28}$$

So the first row of P^2 is:

$$[0.72 \;\; 0.28]$$

Two days later if we start in Sunny

Possible 2-step paths from **Sunny**:

$S \to S \to S$ $\quad 0.8 \times 0.8 = \mathbf{0.64}$

$S \to S \to R$ $\quad 0.8 \times 0.2 = 0.16$

$S \to R \to S$ $\quad 0.2 \times 0.4 = \mathbf{0.08}$

$S \to R \to R$ $\quad 0.2 \times 0.6 = 0.12$

Add the paths that end in the same state:

$$P(\text{Sunny after 2 days}) = 0.64 + 0.08 = \mathbf{0.72}$$

$$P(\text{Rainy after 2 days}) = 0.16 + 0.12 = \mathbf{0.28}$$

So the first row of P^2 is:

$$[0.72 \;\; 0.28]$$

Matrix powers give probabilities after several days

$$P^2 = \begin{bmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{bmatrix} \times \begin{bmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{bmatrix} = \begin{bmatrix} 0.72 & 0.28 \\ 0.56 & 0.44 \end{bmatrix}$$

Starting from **Sunny**:

- Sunny after 2 days: 0.72
- Rainy after 2 days: 0.28

Starting from **Rainy**:

- Sunny after 2 days: 0.56
- Rainy after 2 days: 0.44

What happens as we keep multiplying?

$$\mathbf{P}$$

$$\begin{bmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{bmatrix}$$

$$\mathbf{P}^2 \qquad \mathbf{P}^5 \qquad \mathbf{P}^{10}$$

$$\begin{bmatrix} 0.720 & 0.280 \\ 0.560 & 0.440 \end{bmatrix} \quad \begin{bmatrix} 0.670 & 0.330 \\ 0.660 & 0.340 \end{bmatrix} \quad \begin{bmatrix} 0.667 & 0.333 \\ 0.667 & 0.333 \end{bmatrix}$$

The rows begin to converge to the same distribution:

$$\begin{bmatrix} \frac{2}{3} & \frac{1}{3} \end{bmatrix}$$

What happens as we keep multiplying?

$$\mathbf{P}$$

$$\begin{bmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{bmatrix}$$

$$\mathbf{P}^2 \qquad \mathbf{P}^5 \qquad \mathbf{P}^{10}$$

$$\begin{bmatrix} 0.720 & 0.280 \\ 0.560 & 0.440 \end{bmatrix} \quad \begin{bmatrix} 0.670 & 0.330 \\ 0.660 & 0.340 \end{bmatrix} \quad \begin{bmatrix} 0.667 & 0.333 \\ 0.667 & 0.333 \end{bmatrix}$$

The rows begin to converge to the same distribution:

$$\begin{bmatrix} \frac{2}{3} & \frac{1}{3} \end{bmatrix}$$

What happens as we keep multiplying?

$$\mathbf{P}$$

$$\begin{bmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{bmatrix}$$

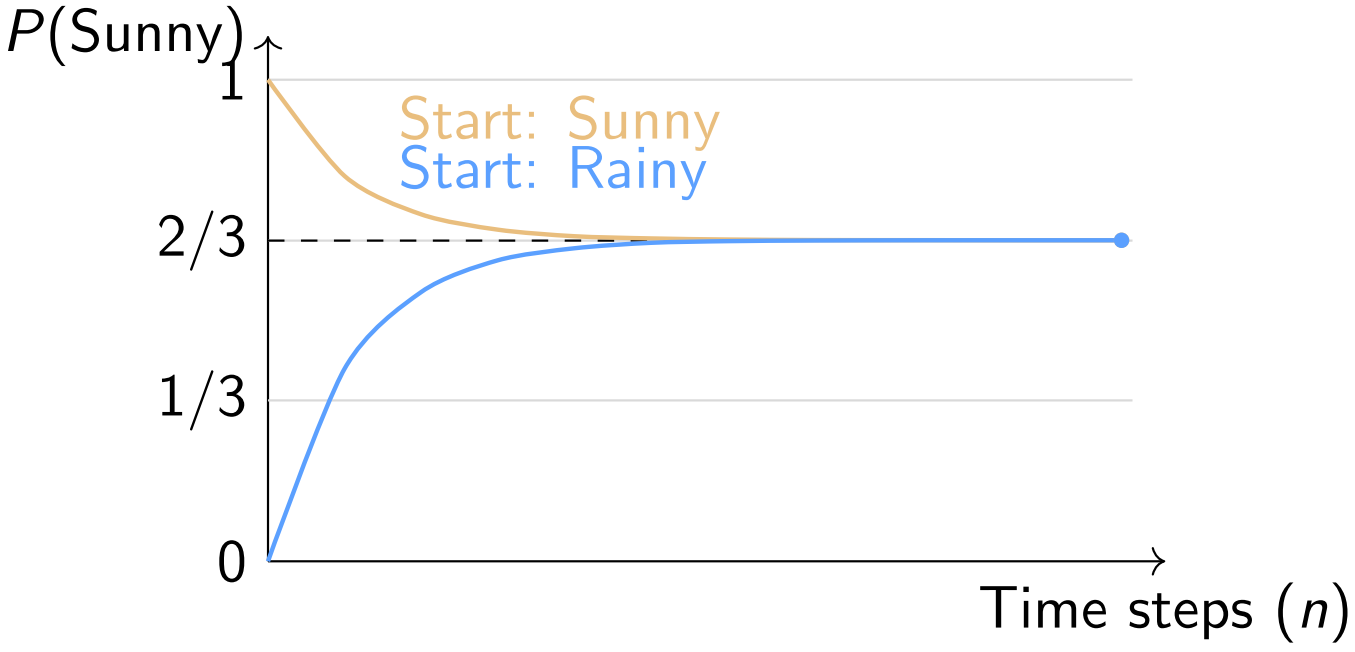
$$\mathbf{P}^2 \qquad \mathbf{P}^5 \qquad \mathbf{P}^{10}$$

$$\begin{bmatrix} 0.720 & 0.280 \\ 0.560 & 0.440 \end{bmatrix} \quad \begin{bmatrix} 0.670 & 0.330 \\ 0.660 & 0.340 \end{bmatrix} \quad \begin{bmatrix} 0.667 & 0.333 \\ 0.667 & 0.333 \end{bmatrix}$$

The rows begin to converge to the same distribution:

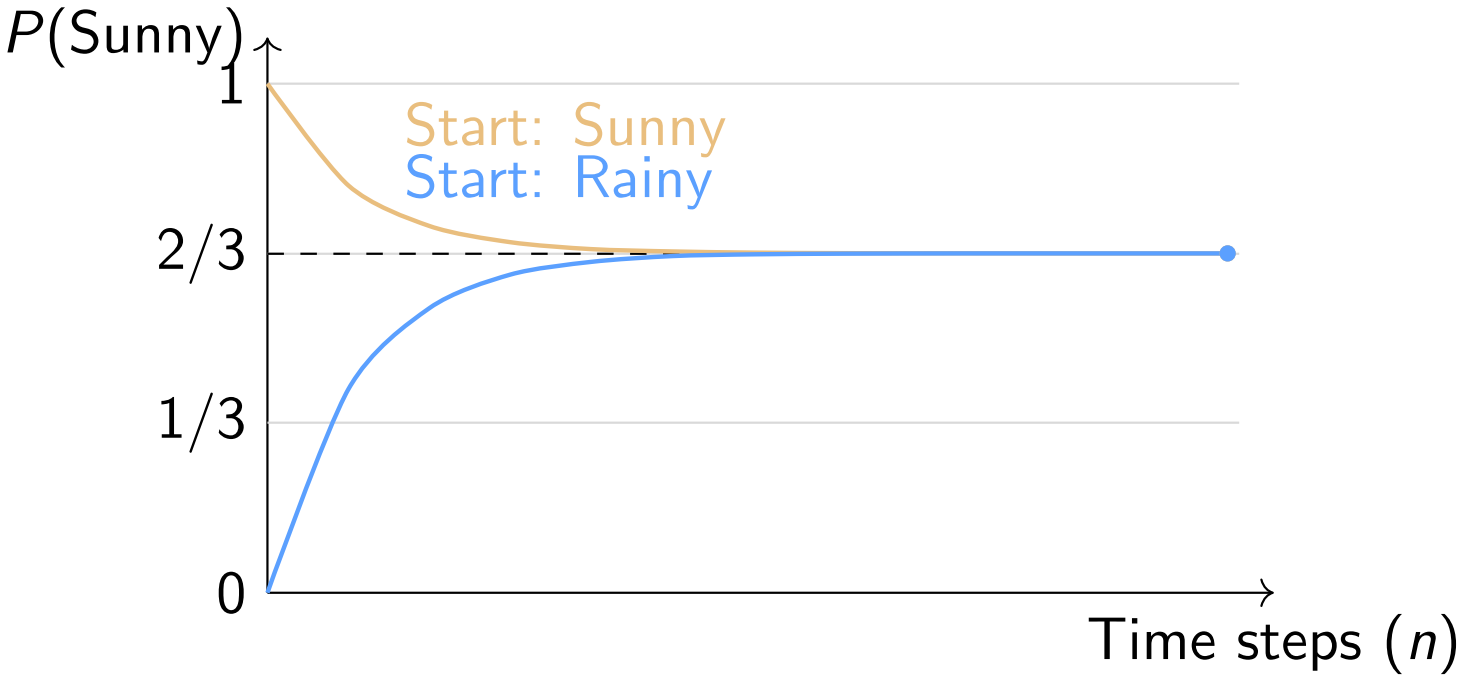
$$\begin{bmatrix} \frac{2}{3} & \frac{1}{3} \end{bmatrix}$$

Stationary distribution



No matter where the system starts, if we wait long enough the probability of being in each state approaches a fixed value that depends only on the transition probabilities, not on the initial conditions.

Stationary distribution



Both starting points converge to

$$\pi = \begin{bmatrix} \frac{2}{3} & \frac{1}{3} \end{bmatrix}$$

Stationary distribution as an equation

$$\pi P = \pi$$

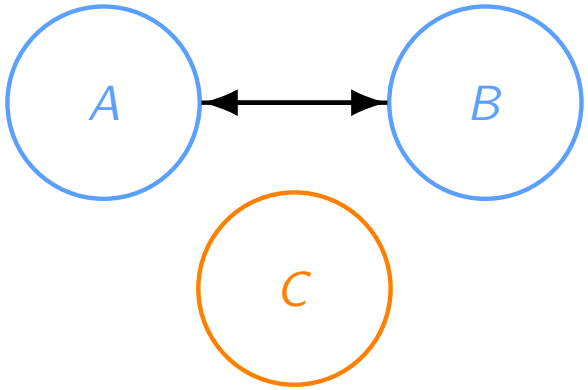
$$\begin{bmatrix} \frac{2}{3} & \frac{1}{3} \end{bmatrix} P = \begin{bmatrix} \frac{2}{3} & \frac{1}{3} \end{bmatrix}$$

$$P = \begin{bmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{bmatrix}$$

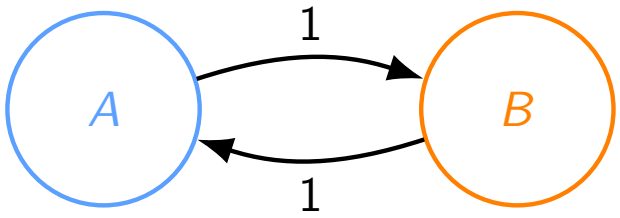
If π is the vector of stationary probabilities, then π times the transition matrix equals π.
In other words, the stationary distribution is unchanged by one step of the chain.

When convergence can fail

Disconnected components



Periodic cycling



For convergence to a unique stationary distribution, we want:

Irreducible

Any state can reach any other state.

Aperiodic

There is no fixed forced cycle.

If both hold, the chain converges from any starting point.

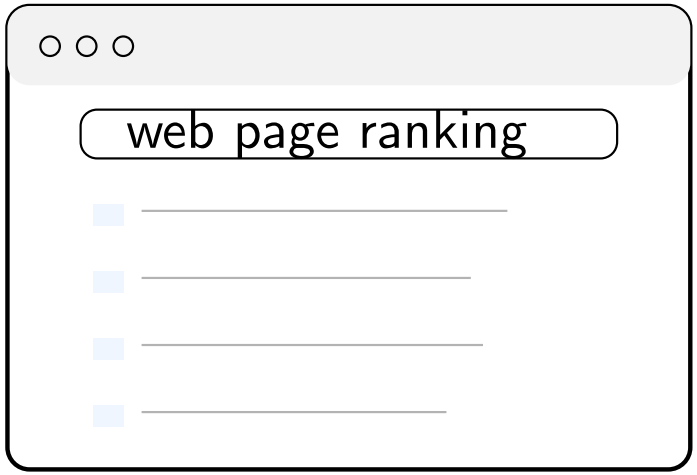
Google's web search revolution

- Search engines must rank billions of pages.
- A page is important if many other pages point to it.
- Markov chains give a natural way to model this idea.

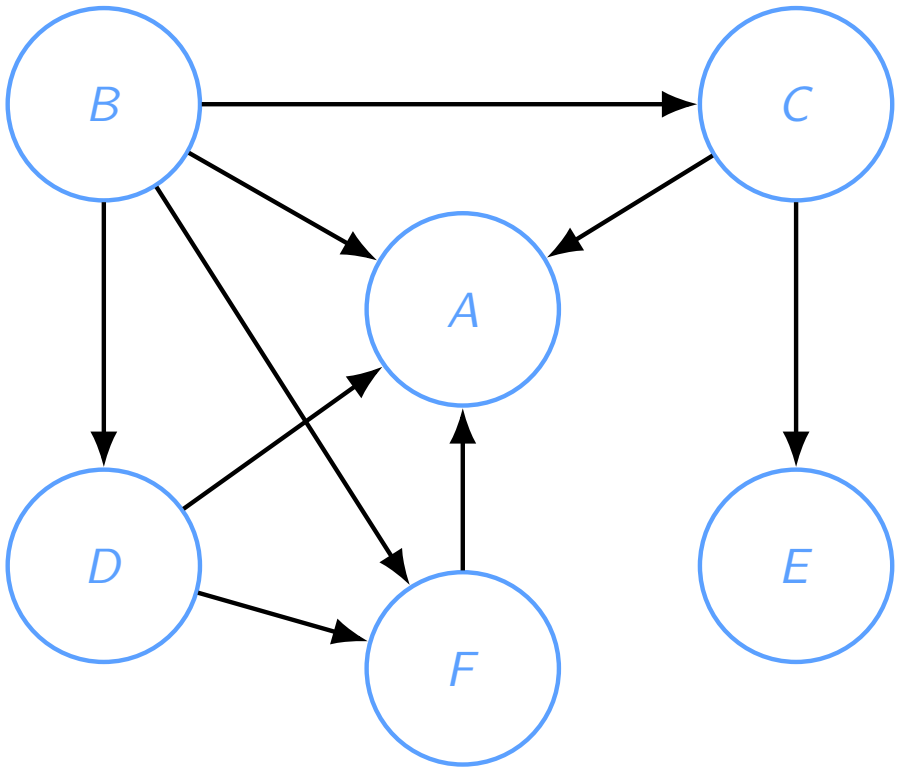
The problem

Question

Billions of pages exist on the web.
Which ones matter most?



PageRank: the random surfer

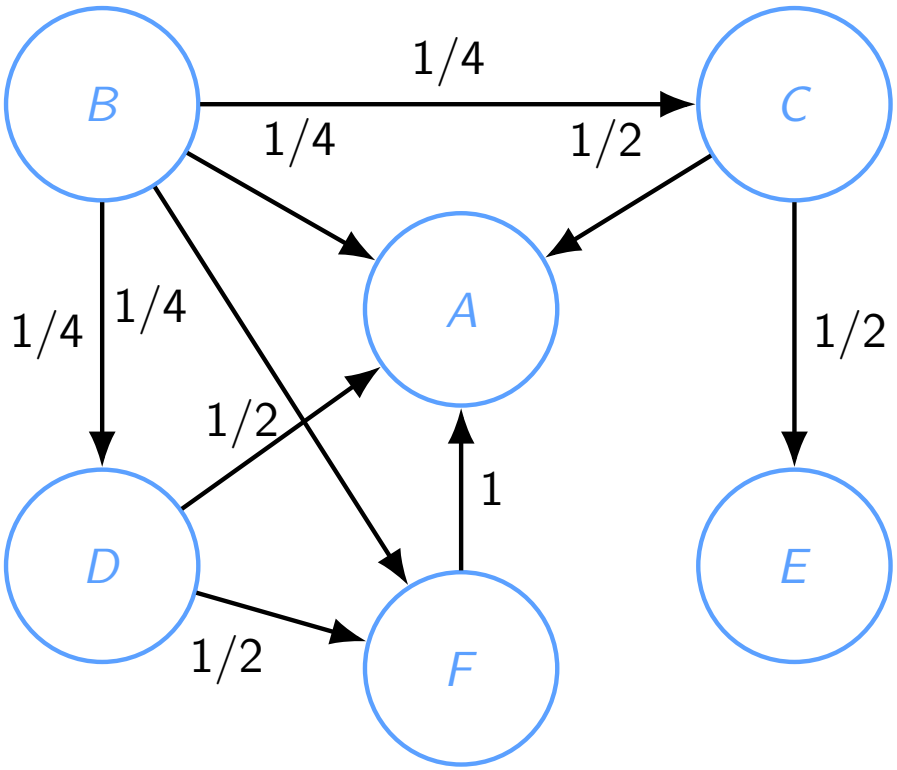


The model

A random surfer moves through the web by clicking links.

- Each page is a state.
- Each link is a transition.
- Following links creates a Markov chain on the web graph.

Transition probabilities on the web graph



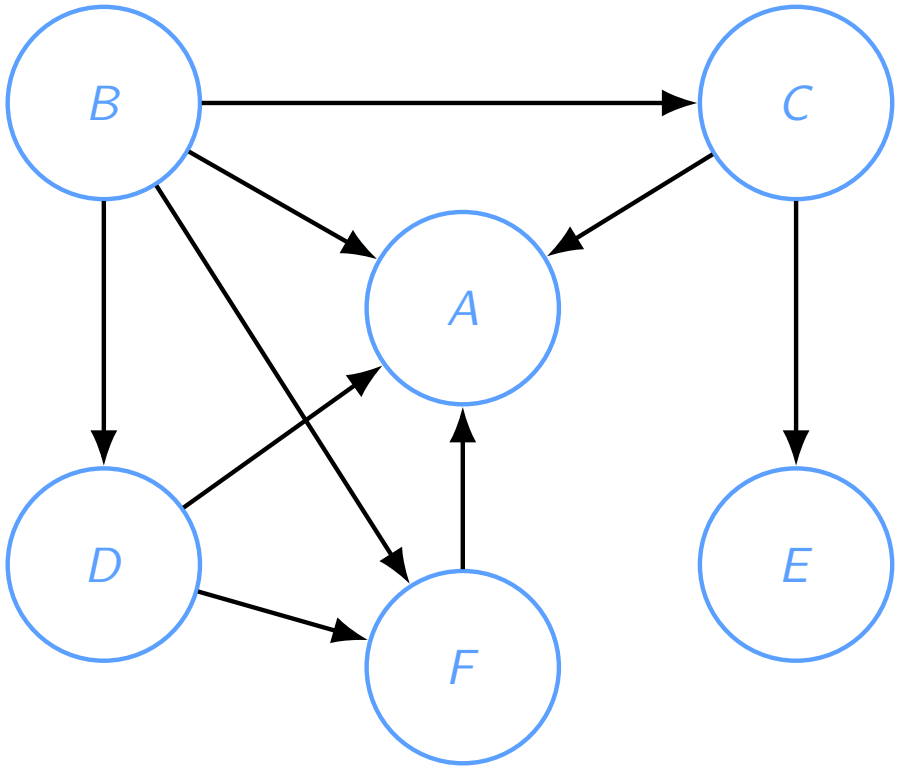
Transition probabilities

If a page has n outgoing links, each one is followed with probability $1/n$.

Example:

- Page B has 4 outgoing links.
- So each one is chosen with probability $1/4$.

What do we want to know?



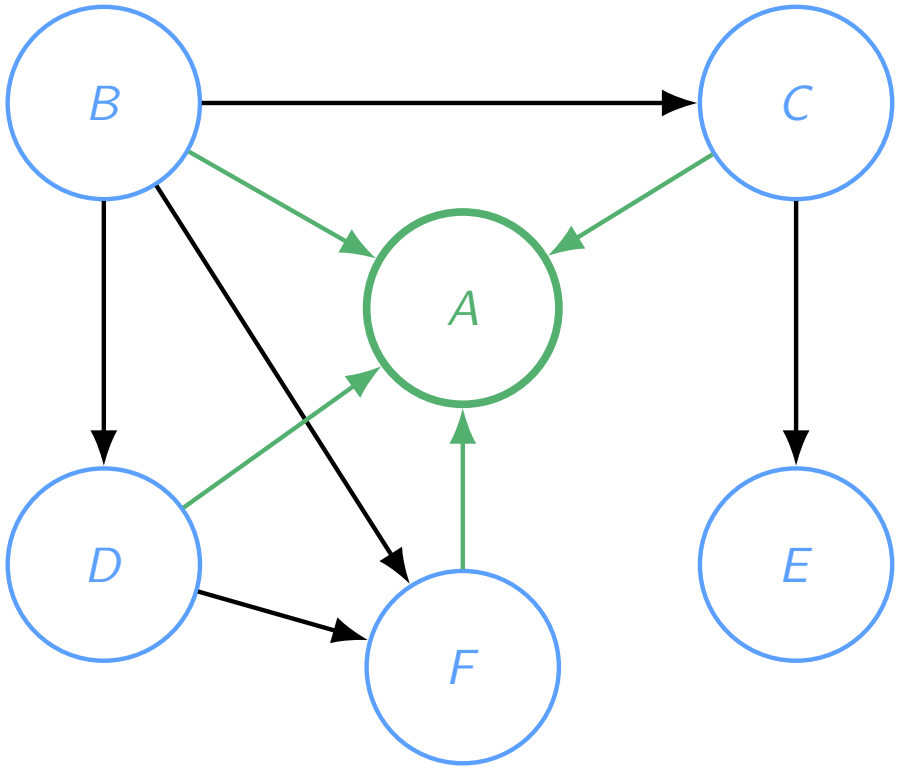
The question

Where does the surfer spend the most time in the long run?

Answer:

The stationary distribution of the Markov chain.

Importance propagates through links



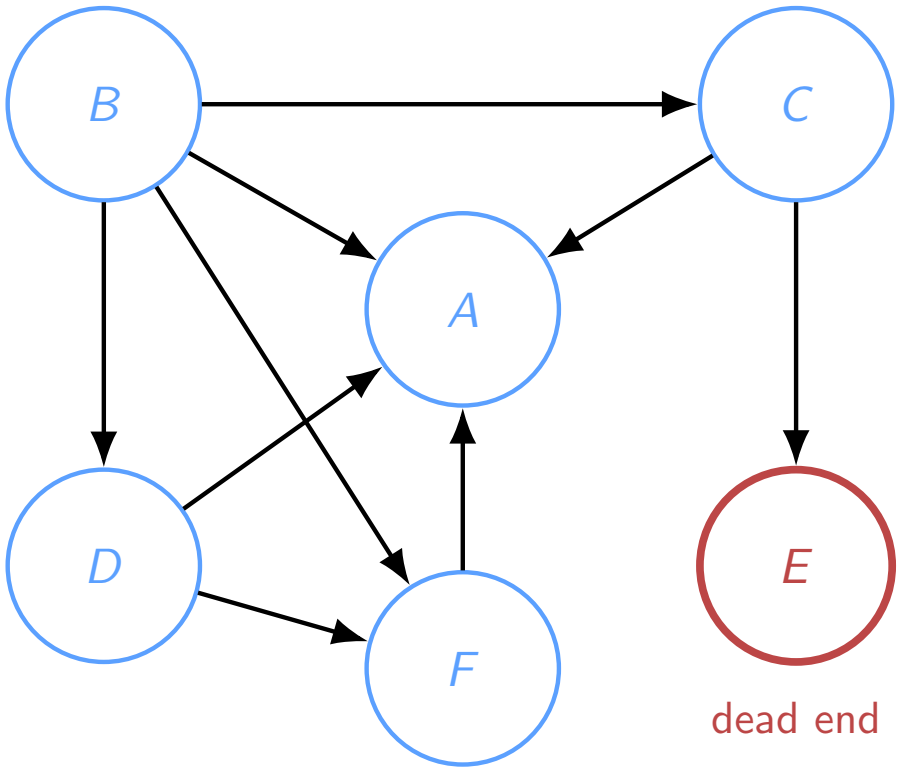
Why PageRank works

- More incoming links lead to more visits.
- Links from important pages matter even more.

Example ranking:

A	0.40
C	0.20
B	0.15
D	0.10
E	0.08
F	0.07

The problem with dead ends



The problem

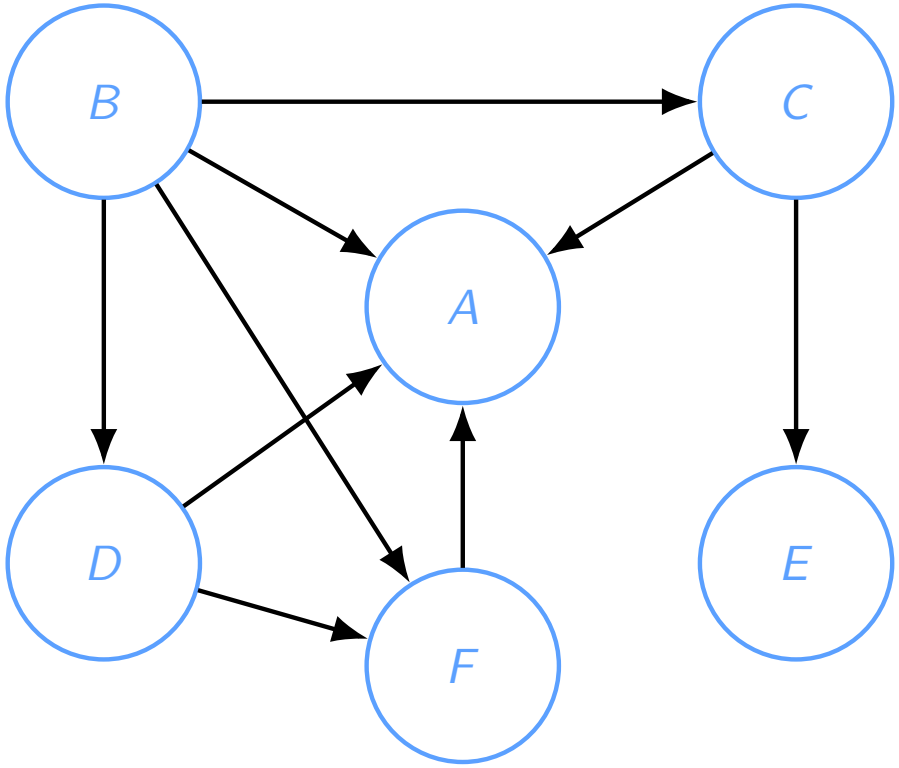
Node E has no outgoing links.

If the surfer arrives there, the process gets stuck.

Why this matters

Dead ends break the simple random-surfer model and make convergence problematic.

The solution: random teleportation



Add teleportation

At each step:

- with probability 0.85, follow a link;
- with probability 0.15, jump to a random page.

This makes the chain irreducible and aperiodic, so a unique stationary distribution exists.
Guarantess convergence to unique stationary distribution.

n-orders Markov models

From Markov chains to text generation

A tiny word-based example

Training text:

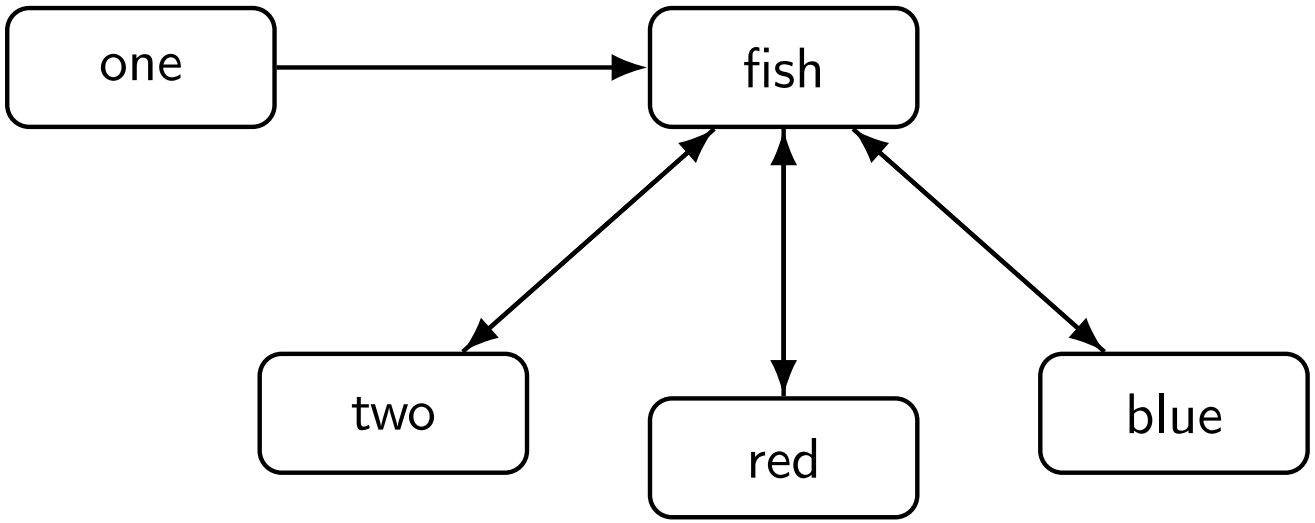
```
one fish two fish red fish blue fish
```

If we use an order-1 word model, the current state is the current word.

The observed transitions are:

```
one  -> fish
fish -> two, red, blue
two  -> fish
red  -> fish
blue -> fish
```

The same example as a graph



If the current state is `fish`, the next word is chosen from `two`, `red`, or `blue`.

One generation from the word model

Start at `one`:

```
one -> fish -> red -> fish -> blue -> fish
```

Output:

```
one fish red fish blue fish
```

Important point:

- We are not copying one fixed sentence.
- We repeatedly choose one outgoing transition from the current state.

Character-based example

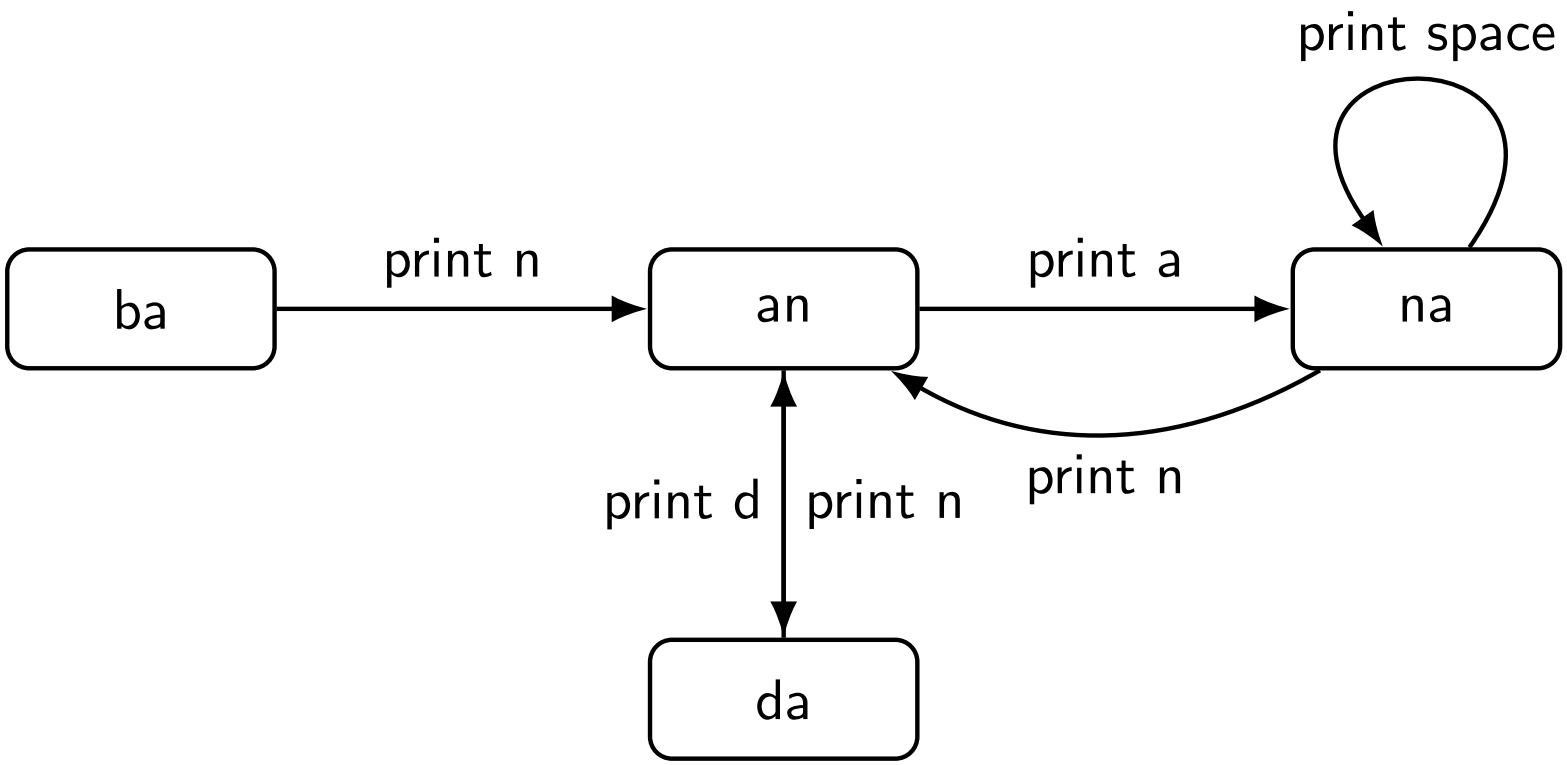
Training text:

```
banana bandana
```

For order $k = 2$, the state is the current block of 2 characters.
Observed transitions:

```
ba -> n
an -> a, d
na -> n, <space>
da -> n
```

Character model as a graph of contexts



Each node is a context. Following an edge prints the next character and updates the context.

From order 1 to order 2

Training text:

```
one fish two fish red fish blue fish
```

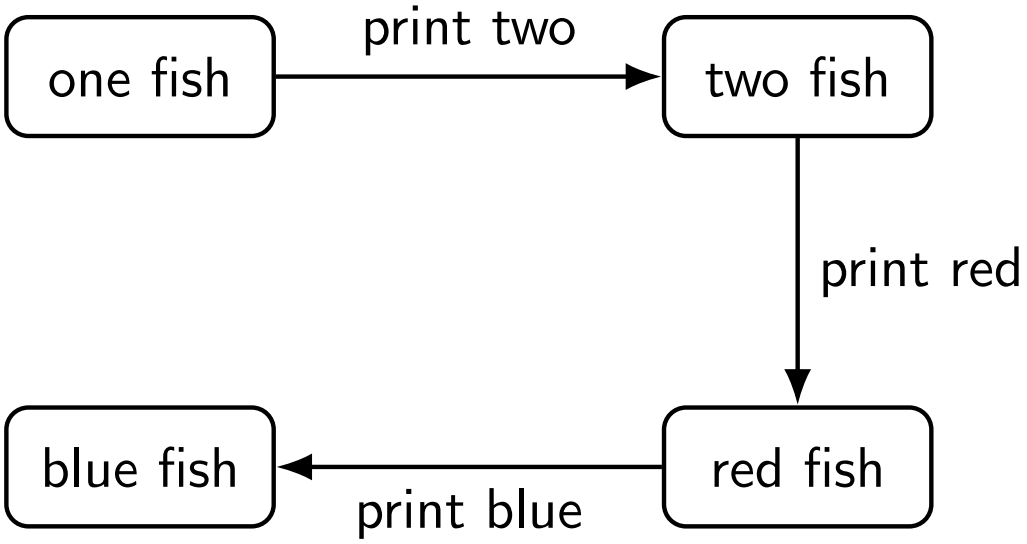
Order 1 uses only the current word:

```
fish -> two, red, blue
```

Order 2 uses two previous words:

```
one fish -> two
two fish -> red
red fish -> blue
```

Order-2 graph: states are now word pairs



Higher order means more memory, more constraint, and usually less randomness.

Effect of the Markov order

Order	Typical effect
low k	more random, more surprising, more nonsense
medium k	locally plausible, often good for toy text generation
high k	copies long fragments from the input; can stop quickly if contexts are rare

In the assignment, the order is given with `-o k`.

Happy Hacking!